

INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO

Ingeniería en Sistemas Computacionales

Docente:

Efrén Rolando Romero León

Materia:

Métodos Numéricos

PROBLEMARIO

Alumnos:

Briana Alexa Del Valle Guzmán **24030220**

Periodo: Enero – Julio

1. REGLA DE LOS 3 PUNTOS (Newton-Cotes abierto)

Fórmula

$$\int_a^b f(x)dx \approx \frac{3h}{2} [f(x_1) + f(x_2) + f(x_3)]$$

Donde x_1, x_2, x_3 son puntos internos

1.1 Caso normal

Planteamiento

$$\int_0^4 (x^2 + 1) dx$$

Resolución

- $h=1$
- Puntos: 1, 2, 3

$$f(1)=2$$

$$f(2)=5$$

$$f(3)=10$$

$$I \approx \frac{3(1)}{2} (2 + 5 + 10) = \frac{3}{2} (17) = 25.5$$

Código

```
import numpy as np

def f(x):
    return x**2 + 1

a, b = 0, 4
h = (b - a) / 4

x1 = a + h
x2 = a + 2*h
x3 = a + 3*h

I = (3*h/2)*(f(x1) + f(x2) + f(x3))
print("Resultado:", I)
Resultado: 25.5

=== Ejecución del código exitosa ===
```

1.2 Mal caso (función oscilante)

Planteamiento

$$\int_0^4 \sin(x) dx$$

Problema: la función cambia rápido → error mayor

Código

```
import numpy as np

def f(x):
    return np.sin(x)

a, b = 0, 4
h = (b - a) / 4

x1 = a + h
x2 = a + 2*h
x3 = a + 3*h

I = (3*h/2)*(f(x1) + f(x2) + f(x3))
```

```
print("Resultado:", I)
Resultado: 2.837832629540168
```

```
=== Ejecución del código exitosa ===
```

1.3 Peor caso (discontinua)

Planteamiento

$$\int_0^4 \frac{1}{x-2} dx$$

Problema: hay **asíntota en x=2** → método falla

Código

```
import numpy as np

def f(x):
    return 1/(x-2)

a, b = 0, 4
h = (b - a) / 4

x1 = a + h
x2 = a + 2*h
x3 = a + 3*h

I = (3*h/2)*(f(x1) + f(x2) + f(x3))
print("Resultado:", I) # puede dar error o infinito
¡ERROR!
Rastreo de la pila (llamada más reciente):
  Archivo "<main.py>", línea 13, en <module>
  Archivo "<main.py>", línea 4, en f
ZeroDivisionError: división de punto flotante por cero

=== El código finalizó con errores ===
```

2. MÉTODO DEL TRAPECIO

Fórmula

$$\int_a^b f(x)dx \approx \frac{b-a}{2} [f(a) + f(b)]$$

2.1 Caso normal

Planteamiento

$$\int_1^3 x^2 dx$$

Resolución

$$I \approx \frac{2}{2}(1^2 + 3^2) = 1(1 + 9) = 10$$

Código

```
def f(x):  
    return x**2  
  
a, b = 1, 3  
I = (b - a)/2 * (f(a) + f(b))  
  
print("Resultado:", I)  
Resultado: 10.0  
  
=== Ejecución del código exitosa ===
```

2.2 Mal caso (curvatura alta)

$$\int_0^2 e^x dx$$

Error alto por curvatura

Código

```
import numpy as np

def f(x):
    return np.exp(x)

a, b = 0, 2
I = (b - a)/2 * (f(a) + f(b))

print("Resultado:", I)
Resultado: 8.38905609893065

=== Ejecución del código exitosa ===
```

2.3 Peor caso (discontinua)

$$\int_0^2 \frac{1}{x} dx$$

Singularidad en 0

Código

```
def f(x):
    return 1/x

a, b = 0, 2
I = (b - a)/2 * (f(a) + f(b))

print("Resultado:", I)
¡ERROR!
Rastreo de la pila (llamada más reciente):
  Archivo "<main.py>", línea 5, en <module>
    Archivo "<main.py>", línea 2, en f
ZeroDivisionError: división por cero

=== El código finalizó con errores ===
```

3. MÉTODO DE SIMPSON 1/3

Fórmula

$$\int_a^b f(x)dx \approx \frac{b-a}{6} [f(a) + 4f(m) + f(b)]$$

3.1 Caso normal

Planteamiento

$$\int_0^2 x^2 dx$$

Resolución

- $m=1$

$$I \approx \frac{2}{6}(0 + 4(1) + 4) = \frac{2}{6}(8) = 2.67$$

Código

```
def f(x):  
    return x**2  
  
a, b = 0, 2  
m = (a + b) / 2  
  
I = (b - a)/6 * (f(a) + 4*f(m) + f(b))  
print("Resultado:", I)
```

Resultado: 2.6666666666666665

=== Ejecución del código exitosa ===

3.2 Mal caso (no suave)

$$\int_0^2 |x - 1| dx$$

No es suave en $x=1$

Código

```
def f(x):  
    return abs(x-1)  
  
a, b = 0, 2  
m = (a + b)/2  
  
I = (b - a)/6 * (f(a) + 4*f(m) + f(b))  
print("Resultado:", I)  
Resultado: 0.6666666666666666  
  
=== Ejecución del código exitosa ===
```

3.3 Peor caso (discontinua)

$$\int_0^2 \tan(x) dx$$

Discontinuidad cerca de $\frac{\pi}{2}$

Código

```
import numpy as np  
  
def f(x):  
    return np.tan(x)  
  
a, b = 0, 2  
m = (a + b)/2  
  
I = (b - a)/6 * (f(a) + 4*f(m) + f(b))  
print("Resultado:", I)
```


Resultado: 1.34819701178603

=== Ejecución del código exitosa ===

4. MÉTODO DE SIMPSON 3/8

Fórmula

$$\int_a^b f(x)dx \approx \frac{3h}{8}[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$$

donde:

$$h = \frac{b-a}{3}$$

- $x_0=a, x_3=b$
- x_1, x_2 son puntos intermedios

4.1 CASO NORMAL (función suave)

Planteamiento

$$\int_0^3 x^2 dx$$

Resolución

- $h=(3-0)/3=1$

Puntos:

$$x_0=0, x_1=1, x_2=2, x_3=3$$

$$f(0)=0, f(1)=1, f(2)=4, f(3)=9$$

$$I \approx \frac{3(1)}{8}[0 + 3(1) + 3(4) + 9]$$

$$I \approx \frac{3}{8}(0 + 3 + 12 + 9) = \frac{3}{8}(24) = 9$$

Código

```
def f(x):  
    return x**2  
  
a, b = 0, 3  
h = (b - a) / 3  
  
x0 = a  
x1 = a + h  
x2 = a + 2*h  
x3 = b  
  
I = (3*h/8) * (f(x0) + 3*f(x1) + 3*f(x2) + f(x3))  
  
print("Resultado:", I)
```

Resultado: 9.0

=== Ejecución del código exitosa ===

4.2 MAL CASO (función con mucha curvatura)

Planteamiento

$$\int_0^3 e^x dx$$

Problema: crece muy rápido → error mayor

Resolución (aproximada)

- mismos puntos: 0, 1, 2, 3

$$f(0) = 1, f(1) = e, f(2) = e^2, f(3) = e^3$$

$$I \approx \frac{3}{8}[1 + 3e + 3e^2 + e^3]$$

(No es exacto → Simpson pierde precisión aquí)

Código

```
import numpy as np

def f(x):
    return np.exp(x)

a, b = 0, 3
h = (b - a) / 3

x0 = a
x1 = a + h
x2 = a + 2*h
x3 = b

I = (3*h/8) * (f(x0) + 3*f(x1) + 3*f(x2) + f(x3))

print("Resultado:", I)
```

Resultado: 19.277831514508783

=== Ejecución del código exitosa ===

4.3 PEOR CASO (discontinuidad / asíntota)

Planteamiento

$$\int_0^3 \frac{1}{x - 1.5} dx$$

Qué pasa

- El método evalúa valores enormes o indefinidos
- Resultado puede ser infinito o error

Código

```
def f(x):  
    return 1/(x - 1.5)  
  
a, b = 0, 3  
h = (b - a) / 3  
  
x0 = a  
x1 = a + h  
x2 = a + 2*h  
x3 = b  
  
I = (3*h/8) * (f(x0) + 3*f(x1) + 3*f(x2) + f(x3))  
  
print("Resultado:", I) # puede fallar o dar infinito
```

```
Resultado: -1.249000902703301e-16
```

```
=== Ejecución del código exitosa ===
```

CONCLUSIÓN

Trapecio → simple pero menos preciso

Simpson → más preciso (usa parábola)

3 puntos → útil cuando no puedes usar extremos

Cuándo fallan:

- Funciones **discontinuas**
- Funciones **muy oscilantes**